control service guide

modifies history

Version	Contributor	Date	Change Log
v1.0	logic.peng	18/08/31	1.add parameters document. 2.add ir api

Summary

In the MINI robot, there is a control board, the robot core board connects the control board through the serial port, Android Application on the core board communicates with the control board using a set of agreed serial port protocols, and controls the steering gear through the control board. Control service is a service application in ROBOT. The main function of control service is to define a unified control board access interface, including: steering control, control board upgrade, infrared and other miscellaneous settings; in addition, in order to allow the robot to perform a complex set of actions, the preset action configuration file (ubx action file) is used in the robot. Inside, the control service has the function of parsing the playing Ubx file and providing the interface to the outside. This document briefly describes these interfaces and SDK integration.

ID distribution of Steering gear

- The Range Of Steering gear Angle

id	body part	rang of Angle	reset angle

id	body part	rang of Angle	reset angle
1	Right Shoulder	0-240	120
2	Right Arm	70-170	120
3	Shoulder Left	0-240	120
4	Left Arm	70-170	120
5	Right Thigh	20-220	120
6	Right Ankle	20-220	120
7	Right foot palms	70-132	120
8	Left thigh	20-220	120
9	Left ankle	20-220	120
10	Left foot palms	70-132	120
11	head	110-130	120
12	chin	110-130	120
13	neck	70-170	120
14	waist	70-170	120

- Reference diagram

  Aaron Swartz

Steering gear control interface

```
//Steering gear info protobuf
message Motor {
   int32 id= 1 ;// id
   int32 upperLimitAngle = 2;//Steering gear upper limit angle，radian unit
   int32 lowerLimitAngle = 3;//Steering gear lower limit angle，radian unit
   int32 servoSener = 4; //Steering gear sensor type: 0- potentiometer 1- magnetic coding
   // Steering gear type: 0-DC core motor 1- DC hollow cup motor 2- BLDC 3-PMSM
   int32 servoMotor = 5;
   // Steering gear MCU type: 0-MEGA8 1-ATSAMD10D14  2-ATSAMC21E15A
   int32 servoMcu = 6;
   int32 servoDriver = 7; // Steering gear Driving chip type: 0-8838 1-6515 2-4959
   int32 servoMode = 8; // Steering gear mode: KG unit
   int32 rigidity = 9; //rigidity
   int32 protectMode = 10; //protect type
   int32 protectLevel = 11;//protect level
   bool protectable = 12;//
   int32 offsetAngle = 13;//offset angle, a angle compensation value of steering gear during motion
   string softwareVersion = 14;//software version
   string hardwareVersion = 15;//hardware version
   int32 servoVolt = 16; //Working voltage of steering gear = ServoVolt*3.7
   int32 resetAngle = 17;//reset angle, a angle when robot standing up
}

//Motion parameters of steering gear
message MotorArgs {
   int32 id = 1;//Steering gear ID
   int32 angle = 2;//Steering gear angle
   int32 runMode = 3;//run mode:0:Uniform velocity，1：variable velocity
   int32 runTime = 4;//time
}

message MotorAngle{
   int32 id = 1;
   int32 angle = 2;
}
message ServoStatus {
```

```
    int32 id = 1;//Steering gear id
    int32 status = 2;
    //The values are as follows:
      0-- steering gear response function has been successfully operated.
      1-- steering engine does not respond to function operation or response timeout;
      2-- steering gear response data error
      3-- The read / write function of steering group structure is not initialized or failed.
      4--Low temperature protection for  steering gear
      5--Over temperature protection of steering gear
      6-- steering gear low voltage protection
      7-- overvoltage protection for steering gear
      8-- steering gear overcurrent protection
      9-- steering torque protection
      10-- actuator fuse dislocation protection
      11-- steering gear stall protection
      12-- steering gear hardware error
}



public class MotorApi {
/**
   * Get robot steering gear information list
   *
   * @param listener ResponseListener with List of Motion.Motor
   */
  public void getMotorList(@NonNull final ResponseListener<List<Motion.Motor>> listener) ;

  /**
   * Get robot steering gear information list
   *
   * @return List of Motion.Motor
   */
  public @Nullable List<Motion.Motor> getMotorList() ;

  /**
   * Read robot steering information
```

```
     *
     * @param id id
     * @return Motion.Motor
     */
    public @Nullable Motion.Motor getMotor(int id) ;

    /**
     * Get the number of steering gear on the robot.
     *
     * @return int
     */
    public int getMotorSum() ;
    /**
     * Move the single steering gear to the specified absolute angle.
     *
     * @param motorId id
     * @param angle target angle
     * @param duration duration of movement
     * @param listener ResponseListener
     */
    public void moveToAbsoluteAngle(@IntRange(from = 1, to = 14) final int motorId, final int angle,
        @IntRange(from = 0, to = 5000) final int duration,
        @Nullable ResponseListener<Void> listener) ;

    /**
     * Move the single steering gear to the specified absolute angle.
     *
     * @param motorId id
     * @param angle target angle
     * @param duration duration of movement
     * @param priority priority
     * @param listener ResponseListener
     */
    public void moveToAbsoluteAngle(@IntRange(from = 1, to = 14) final int motorId, final int angle,
        @IntRange(from = 0, to = 5000) final int duration, Priority priority,
        @Nullable final ResponseListener<Void> listener) ;
```

```
/**
 * Move the single steering gear to the specified absolute angle.
 *
 * @param motorId id
 * @param angle target angle
 * @param duration duration of movement
 * @param runMode uniform or variable velocity
 * @param priority priority
 * @param listener ResponseListener
 */
public void moveToAbsoluteAngle(@IntRange(from = 1, to = 14) final int motorId, final int angle,
    @IntRange(from = 0, to = 5000) final int duration, final RunMode runMode, Priority priority,
    @Nullable final ResponseListener<Void> listener);

/**
 * Move a set of steering gear to a specified absolute angle.
 *
 * @param motorArgs steering movement args
 * @param listener ResponseListener
 */
public void moveToAbsoluteAngle(@NonNull List<Motion.MotorArg> motorArgs,
    @Nullable ResponseListener<Void> listener);

/**
 * Move a set of steering gear to a specified absolute angle.
 *
 * @param motorArgs steering movement args
 * @param listener ResponseListener
 * @param priority priority
 */
public void moveToAbsoluteAngle(@NonNull final List<Motion.MotorArg> motorArgs, Priority priority,
    @Nullable final ResponseListener<Void> listener);

/**
 * Read the current angle of a single steering gear
 *
 * @param motorId id
```

```
     * @param adcump adcump after read angle
     * @param listener ResponseListener
     */
    public void readAbsoluteAngle(final int motorId, boolean adcump,
        @NonNull final ResponseListener<Integer> listener) ;

    /**
     * Read the current angle of a single steering gear
     *
     * @param motorId id
     * @param adcump adcump after read angle
     * @return int >0,steering angle，=-1,error happen
     */
    public int readAbsoluteAngle(final int motorId, boolean adcump);

    /**
     * Read the current angle of a single steering gear
     *
     * @param motorId id
     * @return int >0,steering angle，=-1,error happen
     */
    public int readAbsoluteAngle(final int motorId) ;

    /**
     * Read the current angle of a single steering gear
     *
     * @param motorIds id
     * @param adcump adcump after read angle
     * @param listener response listener
     */
    public void readAbsoluteAngle(@NonNull List<Integer> motorIds, boolean adcump,
        @NonNull final ResponseListener<List<Motion.MotorAngle>> listener);

    /**
     * Read the current angle of a single steering gear
     *
     * @param motorIds ids
```

```
     * @param adcump adcump after read angle
     * @return List of Motion.MotorAngle
     */
    public @Nullable List<Motion.MotorAngle> readAbsoluteAngle(@NonNull List<Integer> motorIds,
        boolean adcump) ;

    /**
     * Read the current angle of a single steering gear
     *
     * @param motorIds ids
     * @return List of Motion.MotorAngle
     */
    public @Nullable List<Motion.MotorAngle> readAbsoluteAngle(@NonNull List<Integer> motorIds) ;

    /**
     * Read the current angle of a single steering gear
     *
     * @param motorIds id
     * @param adcump adcump after read angle
     * @return List of Motion.MotorAngle
     */
    public @Nullable List<Motion.MotorAngle> readAbsoluteAngle(boolean adcump, int... motorIds);

    /**
     * Read the current angle of a single steering gear
     *
     * @param motorIds ids
     * @return List of Motion.MotorAngle
     */
    public @Nullable List<Motion.MotorAngle> readAbsoluteAngle(int... motorIds) ;

    /**
     * Read offset angle of steering gear - use of production line tool for angle calibration of steering gear
     *
     * @param motorId id
     * @param listener listener
     */
```

```
public void readOffsetAngle(final int motorId,
    @NonNull final ResponseListener<Integer> listener) ;

/**
 * Read offset angle of steering gear - use of production line tool for angle calibration of steering gear
 *
 * @param motorId ids
 * @return int < Integer.MAX_VALUE
 */
public int readOffsetAngle(final int motorId) ;

/**
 * Read offset angle of steering gear - use of production line tool for angle calibration of steering gear
 *
 * @param motorIds ids
 * @return List of Motion.MotorAngle
 */
public @Nullable List<Motion.MotorAngle> readOffsetAngle(int... motorIds);

/**
 * Read offset angle of steering gear - use of production line tool for angle calibration of steering gear
 *
 * @param motorIds ids
 * @return List of Motion.MotorAngle
 */
public @Nullable List<Motion.MotorAngle> readOffsetAngle(@NonNull List<Integer> motorIds) ;

/**
 * Read offset angle of steering gear - use of production line tool for angle calibration of steering gear
 *
 * @param motorIds ids
 * @param listener listner
 */
public void readOffsetAngle(@NonNull List<Integer> motorIds,
    @NonNull final ResponseListener<List<Motion.MotorAngle>> listener);

/**
```

```
     * Set the single steering gear calibration angle - the production line tool is used for steering angle calibration
     *
     * @param motorId ids
     * @param offsetAngle new offset angle
     * @return boolean success?
     */
    public boolean setOffsetAngle(final int motorId, final int offsetAngle);

    /**
     * Set up a set of steering gear calibration angle - production line tool used for steering angle calibration
     *
     * @param motorAngles ids
     * @return boolean success?
     */
    public boolean setOffsetAngle(Motion.MotorAngle... motorAngles) ;

    /**
     * Set up a set of steering gear calibration angle - production line tool used for steering angle calibration
     *
     * @param motorAngles a set of new angle
     * @return boolean success?
     */
    public List<Motion.ServoStatus> setOffsetAngle(@NonNull List<Motion.MotorAngle> motorAngles);

    /**
     * Set the single steering gear calibration angle - the production line tool is used for steering angle calibration
     *
     * @param motorId id
     * @param adjustAngle new angle
     * @param listener listener
     */
    public void setOffsetAngle(final int motorId, final int adjustAngle,
        @Nullable final ResponseListener<List<Motion.ServoStatus>> listener);

    /**
     * Set up a set of steering gear calibration angle - production line tool used for steering angle calibration
     *
```

```
     * @param motorAngles a set of motor offset angle
     * @param listener listener
     */
    public void setOffsetAngle(@NonNull List<Motion.MotorAngle> motorAngles,
        @Nullable final ResponseListener<List<Motion.ServoStatus>> listener) ;

    /**
     * Set up a single steering gear reference angle -- production line tool used for steering angle calibration
     *
     * @param motorId id
     * @param BasicAngle new basic angle
     * @param servoSener servo type
     * @return boolean success?
     */
    public boolean setBasicAngle(final int motorId, final int BasicAngle,
        final ServoSener servoSener) ;

    /**
     * Set up a single steering gear reference angle -- production line tool used for steering angle calibration
     *
     * @param motorAngles a set of new angle
     * @return List of Motion.ServoStatus
     */
    public @Nullable List<Motion.ServoStatus> setBasicAngle(Motion.MotorAngle... motorAngles);

    /**
     * Set up a single steering gear reference angle -- production line tool used for steering angle calibration
     *
     * @param motorAngles a set of new angle
     * @return List of Motion.ServoStatus
     */
    public @Nullable List<Motion.ServoStatus> setBasicAngle(
        @NonNull List<Motion.MotorAngle> motorAngles);

    /**
     * Setting up a single steering gear reference angle -- production line tool used for steering angle calibration
     *
```

```
     * @param motorId id
     * @param basicAngle a new basic angle
     * @param servoSener servo type
     * @param listener listener
     */
    public void setBasicAngle(final int motorId, final int basicAngle, final ServoSener servoSener,
        @Nullable final ResponseListener<Boolean> listener) ;

    /**
     * Set up a single steering gear reference angle -- production line tool used for steering angle calibration
     *
     * @param motorAngles a set of new angle
     * @param listener listener
     */
    public void setBasicAngle(@NonNull List<Motion.MotorAngle> motorAngles,
        @Nullable final ResponseListener<List<Motion.ServoStatus>> listener);

    /**
     * lock all steering gear
     *
     * @param listener listener
     */
    public void lockAllMotor(@Nullable ResponseListener<Boolean> listener);

    /**
     * Lock the steering gear: If the steering gear is in motion, the steering gear is locked immediately.
     *  If the steering gear is in power down, the steering gear is locked electrically
     *
     * @param motorId id
     * @param listener listener
     */
    public void lockMotor(final int motorId, @Nullable final ResponseListener<Boolean> listener);

    /**
     * Lock the steering gear: If the steering gear is in motion, the steering gear is locked immediately.
     *  If the steering gear is in power down, the steering gear is locked electrically
     *
```

```
     * @param motorIds id
     * @param listener listener
     * @param priority priority
     */
    public void lockMotor(@NonNull final List<Integer> motorIds, Priority priority,
        @Nullable final ResponseListener<Boolean> listener);

    /**
     * unlock all steering gear
     *
     * @param listener listener of unlock motor
     */
    public void unlockAllMotor(@Nullable ResponseListener<Boolean> listener) ;

    /**
     * unlock steering gear
     *
     * @param motorId id
     * @param listener ResponseListener
     */
    public void unlockMotor(final int motorId, final @Nullable ResponseListener<Boolean> listener) ;

    /**
     * unlock steering gear:  If the steering gear is in motion, the rudder first stops moving and then loses power;
     *  if the steering gear stops locking, the steering gear loses power
     *
     * @param motorIds a set of id
     * @param priority priority
     * @param listener ResponseListener
     */
    public void unlockMotor(@NonNull final List<Integer> motorIds, Priority priority,
        @Nullable final ResponseListener<Boolean> listener) ;

    /**
     * Is the single steering gear working normally
     *
     * @param motorId id
```

```
     * @return boolean success
     */
    public boolean checkServoStatus(final int motorId) ;

    /**
     * Get a set of servo states.
     *
     * @param motorIds a set of new angleid
     * @return List of Motion.ServoStatus
     */
    public @Nullable List<Motion.ServoStatus> checkServoStatus(@NonNull List<Integer> motorIds) ;

    /**
     * Get a set of servo states.
     *
     * @param motorIds a set of id
     * @return List of Motion.ServoStatus
     */
    public @Nullable List<Motion.ServoStatus> checkServoStatus(int... motorIds) ;

    /**
     * Clear servo protection bit
     *
     * @param motorIds a set of id
     */
    public boolean clearProtectFlag(@NonNull List<Integer> motorIds) ;

    /**
     * Clear servo protection bit
     *
     * @param motorIds a set of id
     */
    public boolean clearProtectFlag(int... motorIds);

    /**
     * reset steering gear
     *
```

```
     * @param priority priority
     * @param listener listener
     */
    public void reset(Priority priority, @Nullable final ResponseListener<Boolean> listener);

    /**
     * reset steering gear
     *
     * @param listener listner
     */
    public void reset(@Nullable final ResponseListener<Boolean> listener) ;

    /**
     * Turn off steering gear power
     */
    public void powerOff() ;

    /**
     * Turn on the steering gear
     *
     * @param listener listener of call
     */
    public void powerOn(@Nullable final ResponseListener<Boolean> listener) ;

    /**
     * Turn on the steering gear power synchronously. Do not use it in the main thread
     */
    public void powerOn();

    /**
     * motor status receiver
     *
     * @param receiver Motor Status Receiver
     */
    public void subscribeMotorStatusEvent(MotorStatusReceiver receiver) ;

    /**
```

```
     * motor status unreceiver
     *
     * @param receiver Motor Status Receiver
     */
    public void unsubscribeMotorStatusEvent(MotorStatusReceiver receiver) ;

}
```

control-board subsystem upgrade interface

- Description: Upgrade interface is divided into two types, control board upgrade and steering gear upgrade, if both need to upgrade, first up the chest plate, then the elevator

```
public class UpgradeApi {
    /**
     * Start chest upgrades
     *
     * @param filesize file size
     * @param mode     0:Boot, 1:App - control board is divided into two procedures, Boot is a bootstrap program,
     *                 and App is a program that implements some functions
     * @param listener listener
     */
    public void beginCtrlUpgrade(@IntRange(from = 0, to = 1) int mode, int filesize, @NonNull  ResponseListener<Void> listener);

    /**
     * Writing chest upgrade packets
     *
     * @param pageNum  page num
     * @param pageData page data
     * @param listener listener
     */
    public void writeCtrlUpgradeData(int pageNum, @NonNull byte[] pageData, @NonNull  ResponseListener<Void> listener);

    /**
```

```
    * End chest upgrades
    *
    * @param md5      md5sum
    * @param listener listener
    */
    public void endCtrlUpgrade(@NonNull byte[] md5, @NonNull  ResponseListener<Void> listener);

    /**
    * Start steering gear upgrade
    *
    * @param ids      a set of steering gear ID
    * @param fileSize file size
    * @param listener listener
    */
    public void beginMotorUpgrade(@NonNull List<Integer> ids, int fileSize, @NonNull  ResponseListener<Void> listener);

    /**
    * Write upgrade package to the steering gear
    *
    * @param motorId  id
    * @param islast   is last package
    * @param pageNum  page num
    * @param pageData page data
    * @param listener listener
    */
    public void writeMotorUpgradeData(int motorId, boolean islast, int pageNum, @NonNull byte[] pageData, @NonNull  ResponseListener<Void> lis

}
```

control board subsystem interface

```
public class SysApi {
/**
   * Read robot serial number
```

```
     *
     * @param listener ResponseListener listener
     */
    public void readRobotSid(@NonNull final ResponseListener<String> listener) ;

    /**
     * Read robot serial number
     *
     * @return String sid
     */
    public String readRobotSid() ;

    /**
     * write robot serial number
     *
     * @param sid String new sid
     * @param listener ResponseListener listener
     */
    public void writeRobotSid(@NonNull String sid, @Nullable final ResponseListener<Void> listener) ;

    /**
     * write robot serial number
     *
     * @param sid String new sid
     * @return boolean success
     */
    public boolean writeRobotSid(@NonNull String sid);

    /**
     * Read robot control-board software version
     *
     * @param listener ResponseListener listener
     */
    public void readCtrlVersion(@NonNull final ResponseListener<String> listener) ;

    /**
     * Read robot control-board software version
```

```
     *
     * @return String version
     */
    public String readCtrlVersion() ;

    /**
     * Read robot control-board app version
     *
     * @return String app version
     */
    public String readAppVersion();

    /**
     * read ota version of firmware
     *
     * @param listener ResponseListener listener
     */
    public void readFirmwareVersion(@NonNull final ResponseListener<String> listener) ;

    /**
     * read ota version of firmware
     *
     * @return String ota version
     */
    public String readFirmwareVersion() ;

    /**
     * set ota version of firmware
     *
     * @param version firmware version
     * @param listener ResponseListener listener
     */
    public void setFirmwareVersion(@NonNull String version,
        @NonNull final ResponseListener<Boolean> listener);

    /**
     * set ota version of firmware
```

```
     *
     * @param version ota version
     * @return boolean
     */
    public boolean setFirmwareVersion(@NonNull String version) ;

    /**
     * shutdown control-board subsystem
     */
    public void shutdown();

    /**
     * startup control-board subsystem: subsystem startup need 1500ms at least
     *
     * @param listener {@link Response Listener}
     */
    public void startup(@NonNull final ResponseListener<Void> listener) ;

    /**
     * is control-board subsystem started?
     *
     * @return boolean
     */
    public boolean isStarted() ;

    /**
     * subsystem error receiver
     *
     * @param receiver Subsystem Error Receiver
     */
    public void subscribeSubsystemErrorEvent(SubsystemErrorReceiver receiver) ;
    /**
     * subsystem error unreceiver
     *
     * @param receiver Subsystem Error Receiver
     */
    public void unsubscribeSubsystemErrorEvent(SubsystemErrorReceiver receiver) ;
```

```
}
```

action interface

```
//action info
message Action {
    string id = 1; //id
    string cnName = 2; //cn name
    string enName = 3; //en name
    string desc = 4; //description
    int32 time = 5; //duration
    int32 type = 6; //
    bool unsafeAction = 7; //
}
```

```
public class ActionApi {
  /**
   * Asynchronous access to system built in action files list
   *
   * @param listener ResponseListener
   */
  public void getActionList(@NonNull final ResponseListener<List<Motion.Action>> listener);

  /**
   * Get the list of action files supported by the system
   *
   * @return list of Motion.Action
   */
  public @Nullable List<Motion.Action> getActionList();

  /**
```

```
 * Get a list of custom action files built into sdcard/customize/actions on the robot
 *
 * @return list of {@link Motion.Action}
 */
public List<Motion.Action> getCustomizeActionList();

/**
 * Get a action information built in robot by id
 *
 * @param actionId action id
 * @return {@link Motion.Action}
 */
public @Nullable Motion.Action getActionInfo(String actionId);

/**
 * Get a action information built in the /sdcard/sdcard/customize directory
 *
 * @param actionId action id
 * @return {@link Motion.Action}
 */
public @Nullable Motion.Action getCustomizeActionInfo(String actionId);

/**
 * Play a system built-in action
 *
 * @param action String
 * @param listener ResponseListener
 */
public void playAction(@NonNull final String action,
    @Nullable final ResponseListener<Void> listener);

/**
 * Play a system built-in action
 *
 * @param action String
 * @param priority priority
 * @param listener ResponseListener
```

```
     */
    public void playAction(@NonNull final String action, Priority priority,
        @Nullable final ResponseListener<Void> listener);

    /**
     * Play a custom action built into sdcard/customize/actions
     *
     * @param actionId action Id
     * @param listener ResponseListener
     */
    public void playCustomizeAction(@NonNull final String actionId,
        @Nullable final ResponseListener<Void> listener);

    /**
     * Play a custom action built into sdcard/customize/actions
     *
     * @param actionId action Id
     * @param priority priority
     * @param listener ResponseListener
     */
    public void playCustomizeAction(@NonNull final String actionId, Priority priority,
        @Nullable final ResponseListener<Void> listener);

    /**
     * stops the current action
     *
     * @param listener ResponseListener
     */
    public void stopAction(@Nullable final ResponseListener<Void> listener);

    /**
     * stops the current action
     *
     * @return boolean
     */
    public boolean stopAction();
```

```
/**
 * stops the action by name
 *
 * @param name action name
 */
public void stopActionByName(@NonNull String name) ;

/**
 * is action playing
 *
 * @param listener ResponseListener
 */
public void isPlaying(@NonNull final ResponseListener<Boolean> listener);

/**
 * is action playing
 *
 * @return boolean
 */
public boolean isPlaying();

/**
 * is dangerous action
 *
 * @param actionId action id
 * @return boolean
 */
public boolean unsafeAction(@NonNull String actionId);

/**
 * Determine whether there is a dangerous action in the execution of the action. If no action is played, return to false
 *
 * @return boolean
 */
public boolean unsafeAction() ;

/**
```

```
    * Gets the action information that is being executed at the moment. If there is no action at present, return to null
    *
    * @return Motion.Action of list
    */
   public @Nullable List<Motion.Action> currentAction() ;

   /**
    * action finished receiver
    *
    * @param receiver Action Stopped Receiver
    */
   public void subscribeEvent(ActionStoppedReceiver receiver);

   /**
    * action finished Action Stopped Receiver
    *
    * @param receiver 状态监听
    */
   public void unsubscribeEvent(ActionStoppedReceiver receiver) ;

}

/**
 * action error code
 */
public class ActionErrorCode {
   public static final int INTERRUPTED_BY_STOP = 10001;//action interrupted by stop call
   public static final int INTERRUPTED_BY_BROKEN = 10002;//action interrupted by motor broken
   public static final int INTERRUPTED_BY_PROTECTED = 10003;//action interrupted by motor protected status
   public static final int INTERRUPTED_BY_ACTION = 10004;//action interrupted by new action conflict
   public static final int FILE_NOT_FOUNDED = 10005;//action resource not found
   public static final int DATABASE_EXCEPTION = 10006;//action provider error
   public static final int UBX_PARSE_ERROR = 10008;//action parse failure
   public static final int UBX_PLAY_ERROR = 10009;//action play error
   public static final int INTERRUPTED_BY_STOP_SELF = 10010;//action interrupted by stopActionByName call

}
```

Extended Action Interface

```
public class ActionExApi {
 /**
  * walk Forward
  *
  * @param step step
  * @param priority priority
  * @param listener {@link ActionExListener}
  */
 public void walkForward(final int step, Priority priority, final ActionExListener listener) ;

 /**
  * walk Backward
  *
  * @param step step
  * @param priority priority
  * @param listener {@link ActionExListener}
  */
 public void walkBackward(final int step, final Priority priority,
    final ActionExListener listener) ;

 /**
  * turn Right
  *
  * @param amount amount
  * @param priority priority
  * @param listener {@link ActionExListener}
  */
 public void turnRight(final int amount, final Priority priority,
    final ActionExListener listener) ;

 /**
  * turn Left
```

```
     *
     * @param amount amount
     * @param priority priority
     * @param listener {@link ActionExListener}
     */
    public void turnLeft(final int amount, final Priority priority, final ActionExListener listener) ;

    /**
     * make Bows
     *
     * @param amount amount
     * @param priority priority
     * @param listener {@link ActionExListener}
     */
    public void makeBows(final int amount, Priority priority, final ActionExListener listener) ;

    /**
     * make Nods
     *
     * @param amount amount
     * @param priority priority
     * @param listener {@link ActionExListener}
     */
    public void makeNods(final int amount, final Priority priority, final ActionExListener listener) ;

    /**
     * shake Heads
     *
     * @param amount amount
     * @param priority priority
     * @param listener {@link ActionExListener}
     */
    public void shakeHeads(final int amount, Priority priority, final ActionExListener listener) ;

    /**
     * slanting Heads
     *
```

```
     * @param amount amount
     * @param priority priority
     * @param listener {@link ActionExListener}
     */
    public void slantingHeads(final int amount, Priority priority, final ActionExListener listener) ;

    /**
     * shake Hands
     *
     * @param amount amount
     * @param priority priority
     * @param listener {@link ActionExListener}
     */
    public void shakeHands(final int amount, Priority priority, final ActionExListener listener) ;

    /**
     * wave Hands
     *
     * @param amount amount
     * @param priority priority
     * @param listener {@link ActionExListener}
     */
    public void waveHands(final int amount, Priority priority, final ActionExListener listener) ;

    /**
     * The robot is standing posture, doing push ups, calling the interface
     *
     * @param amount amount
     * @param priority priority
     * @param listener {@link ActionExListener}
     */
    public void makePressUps(final int amount, final Priority priority,
        final ActionExListener listener) ;

    /**
     * stop Walk Forward
     */
```

```
public void stopWalkForward() {
  stopParamActionByName("walkForward");
}

/**
 * stop Walk Backward
 */
public void stopWalkBackward() {
  stopParamActionByName("walkBackward");
}

/**
 * stop Turn Right
 */
public void stopTurnRight() {
  stopParamActionByName("turnRight");
}

/**
 * stop Turn Left
 */
public void stopTurnLeft() {
  stopParamActionByName("turnLeft");
}

/**
 * stop Make Bows
 */
public void stopMakeBows() {
  stopParamActionByName("makeBows");
}

/**
 * stop Make Nods
 */
public void stopMakeNods() {
  stopParamActionByName("makeNods");
```

```
}

/**
 * stop Shake Heads
 */
public void stopShakeHeads() {
  stopParamActionByName("shakeHeads");
}

/**
 * stop Slanting Heads
 */
public void stopSlantingHeads() {
  stopParamActionByName("slantingHeads");
}

/**
 * stop Shake Hands
 */
public void stopShakeHands() {
  stopParamActionByName("shakeHands");
}

/**
 * stop Wave Hands
 */
public void stopWaveHands() {
  stopParamActionByName("waveHands");
}

/**
 * stop Make PressUps
 */
public void stopMakePressUps() {
  stopParamActionByName("makePressUps");
}
```

```
    /**
     * Stop Parameterization by name
     *
     * @param name param action key {makePressUps, waveHands, shakeHands, slantingHeads,shakeHeads,
     *  makeNods,makeBows,turnLeft,turnRight,walkBackward,walkForward }
     */
    public void stopParamActionByName(String name) ;

}

/**
 * Extend Action Listener
 */
public interface ActionExListener {
    void onActonStarted();

    /**
     * action progress
     *
     * @param progress progress
     * @param totalCount total count
     */
    void onActionProgress(int progress, int totalCount);

    void onActionCompleted();

    /**
     * action failure
     *
     * @param errCode error Code {@link ActionErrorCode.Type}
     * @param errMsg error message
     */
    void onActionFailure(@ActionErrorCode.Type int errCode, @NonNull String errMsg);
}
```

Infrared Interface

```
public class IrApi {
/**
   * Infrared calibration -- use of production line tools
   *
   * @param mode calibration mode, 0, 1
   * @return boolean
   */
  public boolean adjust(@IntRange(from = 0, to = 1) int mode);

  /**
   * Infrared detection distance: 50-1500 mm, if -1 indicates failure
   *
   * @return int
   */
  public int distance();

  /**
   * Infrared power power on
   *
   * @return boolean
   */
  public boolean powerOn();

  /**
   * Infrared power power off
   *
   * @return boolean
   */
  public boolean powerOff();

}
```